

Scaling Multi-Robot Search and Rescue: A Comparative Study of Metaheuristic Exploration Methods

Diana Elzeftawy¹, Ethar Hany¹, Jana Saad¹, Mariam Gafar¹, Mennah M. Shaker¹, Nabila Ataby¹,
Jessica Magdy^{2,3} and Omar M. Shehata^{2,3}

¹Computer Science Department, Faculty of Media Engineering and Technology, German University in Cairo, Egypt

²Mechatronics Engineering Department, Faculty of Engineering and Materials Science, German University in Cairo, Egypt

³Multi-Robot Systems (MRS) Research Group, Cairo, Egypt

{diana.elzeftawy, ethar.elsayed, jana.saad, mariam.gafar, mennah.mohamed, nabila.ataby}@student.guc.edu.eg

{jessica.gergis, omar.mohamad}@guc.edu.eg

Abstract—Autonomous multi-robot systems provide a safer alternative to human responders in hazardous disaster environments. This work evaluates a multi-objective grid-based exploration strategy that maximizes coverage while minimizing redundancy, exploration time, and constraint violations. Four metaheuristic algorithms—Simulated Annealing (SA), Genetic Algorithm (GA), Particle Swarm Optimization (PSO), and Aquila Optimization Algorithm (AOA)—are assessed across three environments of increasing complexity. While AOA performs best in small-scale settings, its effectiveness declines as the search space grows. SA and PSO achieve competitive coverage in medium scenarios but struggle with refinement and penalty avoidance at larger scales. Overall, GA exhibits the strongest scalability, maintaining a superior balance between coverage and constraints in complex environments, indicating that population-based evolutionary strategies are more robust for large-scale disaster response.

Index Terms—Multi-robot exploration, metaheuristic optimization, simulated annealing, genetic algorithm, particle swarm optimization, aquila optimization algorithm, path planning, search and rescue, multi-objective optimization.

I. INTRODUCTION

In disaster scenarios such as collapsed structures, underground mines, or industrial fires, the first 72 hours—often referred to as the “Golden Hour”—are critical for survivor recovery. However, these environments are frequently too unstable or toxic for human responders [1]. To mitigate these risks, autonomous robotic swarms are increasingly employed to map hazardous zones and locate victims, leveraging metaheuristic optimization techniques that have proven effective across diverse engineering domains [2]. Multi-robot exploration provides a safer and more efficient alternative by enabling teams of robots to collaboratively generate maps and locate victims, achieving faster coverage and reducing the likelihood of missing critical areas compared to single-agent approaches [3].

The primary objective of this study is to evaluate and compare metaheuristic strategies that enable robotic swarms to

navigate such environments. The optimization problem targets three objectives:

- **Maximize Coverage:** Visiting the maximum number of unique grid cells.
- **Minimize Time:** Reducing total exploration duration.
- **Minimize Redundancy:** Limiting overlapping paths to improve efficiency.

The environment is modeled as a 2D grid with strict constraints, including boundary limits, collision avoidance, and maximum travel distances. To address the resulting trajectory planning problem, this paper implements and compares four metaheuristic algorithms—Simulated Annealing (SA), Genetic Algorithm (GA), Particle Swarm Optimization (PSO), and the Aquila Optimization Algorithm (AOA). These methods are evaluated across varying grid sizes to analyze their scalability and robustness, with particular focus on the performance trade-offs between population-based approaches (GA, PSO, AOA) and the trajectory-based method (SA) as search space complexity increases.

II. RELATED WORK

Multi-robot exploration in disaster environments is a critical application of collaborative robotics, requiring both efficient area coverage and collision avoidance. Recent studies have increasingly moved toward metaheuristic optimization to handle the computational complexity of these large search spaces [4], [5]. Recent work has demonstrated the effectiveness of task allocation methods that handle dynamic deadlines and resource constraints in disaster scenarios [6].

Swarm intelligence approaches have shown significant promise. Paez et al. [7] proposed a Distributed Particle Swarm Optimization (DPSO) algorithm where robots are controlled by virtual forces attracted to targets and repelled by obstacles to minimize search time while avoiding collisions. To address environments with unknown sizes and limited communication, Kashyap et al. [8] introduced a hybrid approach combining Lévy Flight for broad exploration with PSO’s social

component for inter-robot repulsion, which showed improved robustness more than standard methods.

Beyond standard PSO, other nature-inspired algorithms have been adapted for this domain. Romeh and Mirjalili [5] developed the Coordinated Multi-Robot Exploration-Salp Swarm Algorithm (CME-SSA), which optimizes a utility-cost function to maximize the percentage of area explored.

Energy constraints have also become a central focus. Bendahmane and Tlemsani [9] proposed a Modified Butterfly Optimization Algorithm (xBOA) that introduces a crossover operator to balance exploration efficiency with energy consumption. Similarly, Singh et al. [10] presented an energy-aware cooperative exploration model using GA, while recent comparative optimization studies for UGV coordination further highlight the frequent use of GA and Simulated Annealing as baseline metaheuristics in multi-agent routing problems [11], [12].

This paper builds upon these foundations by performing a direct comparative analysis of four distinct classes of metaheuristics, which are Simulated Annealing, Genetic Algorithm, Particle Swarm Optimization and Aquila Optimization Algorithm, to evaluate their scalability and constraint-handling capabilities in increasingly complex grid scenarios.

III. PROBLEM FORMULATION

This study models the Search and Rescue (SAR) mission as a multi-objective optimization problem within a two-dimensional grid environment. A team of N autonomous robots must explore a grid defined by dimensions $X_{max} \times Y_{max}$ to locate victims while minimizing redundant movements and energy consumption.

The optimization problem aims to determine the optimal set of trajectories, where each robot's path is a sequence of discrete waypoints. The system performance is evaluated using a multi-objective fitness function F that balances three competing objectives.

A. Objectives

The performance of the exploration strategy is quantified by the following metrics:

Let N_{unique} denote the number of unique grid cells visited at least once by the robot team. Let N_{visits} denote the total number of grid cell visits accumulated by all robots, including repeated visits to the same cell.

1) *Maximize Coverage* (f_1): Let N_{unique} denote the number of unique grid cells visited by the robot team, and N_{total} denote the total number of grid cells in the environment.

The coverage objective is defined as:

$$f_1 = \frac{N_{unique}}{N_{total}} \quad (1)$$

2) *Minimize Exploration Time* (f_2): Since all robots are assumed to move with a constant velocity, the time required to traverse a path is directly proportional to its length. As the robots operate in parallel, the overall exploration time is therefore governed by the robot that follows the longest

path. Consequently, the objective is to minimize the maximum distance traveled by any single robot:

$$f_2 = \max_{i \in 1, \dots, N} \left(\sum_{t=1}^{T-1} |p_i(t+1) - p_i(t)| \right) \quad (2)$$

where $p_i(t)$ represents the position of robot i at time t . For the final fitness calculation, this value is normalized (f_{2n}) to maintain scale consistency.

3) *Minimize Redundancy* (f_3): To prevent inefficient overlapping paths, redundancy is defined as the ratio of revisited cells to the total number of unique cells visited:

$$f_3 = \frac{N_{visits} - N_{unique}}{N_{visits}} \quad (3)$$

Repeated visits are explicitly counted, as revisiting cells still incurs movement cost and coordination overhead in multi-robot exploration [5].

Unlike approaches that only consider unique cell visitation, this work explicitly accounts for repeated cell visits when evaluating redundancy. Cells revisited by one or multiple robots are counted as valid visits, as they still contribute to exploration effort, energy consumption, and coordination overhead. This formulation provides a more realistic assessment of multi-robot exploration efficiency in constrained environments [5], [10].

B. Fitness Function

The weighted fitness function F aggregates these objectives while applying a penalty P for constraint violations:

$$F = w_1 f_1 - w_2 f_{2n} - w_3 f_3 - P \quad (4)$$

where w_1, w_2, w_3 are positive weighting coefficients. The penalty term P aggregates violations of boundary constraints, collision avoidance, and maximum travel distance, and is scaled to dominate the fitness value when feasibility is violated. The weighting coefficients w_1, w_2 , and w_3 were kept constant across all scenarios to ensure consistency and fairness in comparative evaluation. Since the primary objective of this study is to analyze the scalability and robustness of different metaheuristic algorithms, varying the weights across environments could introduce preference-driven bias and confuse the interpretation of algorithmic performance.

C. Constraints

To ensure feasibility and safety, the system enforces the following constraints:

- **Boundary Constraint:** Robots must remain within grid limits ($0 \leq x_i(t) \leq X_{max}$ and $0 \leq y_i(t) \leq Y_{max}$).
- **Maximum Travel Distance:** The total distance D_i traveled by any single robot must not exceed a predefined limit D_{max} .
- **Collision Avoidance:** The distance between any two robots i and j must remain greater than a safety threshold d_{safe} at all times:

$$\|p_i(t) - p_j(t)\| \geq d_{safe}, \quad \forall i \neq j \quad (5)$$

Similar grid-based formulations and coverage objectives have been widely adopted in multi-robot exploration literature [4], [5], [7].

IV. METHODOLOGY

To solve the formulated problem, we implemented and compared four metaheuristic algorithms alongside a deterministic greedy baseline. A key methodological decision was the solution representation: rather than optimizing every step of a continuous path, we optimized a sparse set of k waypoints per robot (e.g., 8–12 waypoints). These waypoints are interpolated using Bresenham's algorithm to generate the full high-resolution grid path required for accurate fitness evaluation. [13]

A. Simulated Annealing (SA)

Simulated Annealing is a probabilistic method that mimics the metal annealing process. It avoids local optima by accepting worse solutions with a probability that decreases over time.

- **Neighborhood Generation:** New candidate solutions are generated by applying a “Local Waypoint Shift.” A random robot and random waypoint are selected, and the coordinates are modified by small random values constrained within the grid boundaries.
- **Cooling Schedule:** The acceptance probability is governed by a geometric cooling schedule:

$$T_{\text{cur}} = T_0 \alpha^{i_{\text{cur}}}$$

where T_0 is the initial temperature, α is the cooling factor ($0 < \alpha < 1$), and i_{cur} is the current iteration number. [14]

B. Genetic Algorithm (GA)

The Genetic Algorithm mimics biological evolution through selection, crossover, and mutation operators.

- **Selection:** We employed an elitist strategy where the top 10% of the population (the elite solutions) are passed directly to the next generation.
- **Crossover:** A “Segment-Swap Crossover” was implemented instead of standard arithmetic crossover. This method exchanges entire valid path segments between parent solutions.
- **Mutation:** To maintain diversity, a “Strong Waypoint Mutation” is applied to the worst-performing 10% of the population. This adds a random offset δ to waypoint coordinates to push the algorithm out of local optima. [15]

C. Particle Swarm Optimization (PSO)

PSO was introduced by Kennedy and Eberhart in 1995 and is well-suited for continuous waypoint optimization due to its rapid convergence properties. In this implementation, each particle represents the complete set of waypoints for all N robots.

The particle velocity and position are updated using the standard PSO update rule, which combines inertia, cognitive, and social components based on the particle's personal best

and the global best solutions [16]. Random vectors uniformly distributed in $[0, 1]$ introduce stochasticity into the search process.

Parameter Settings: Based on sensitivity analysis, we utilized a swarm size of 50, cognitive and social coefficients $c_1 = c_2 = 1.5$, and an inertia weight $w = 0.7$ to balance global exploration with local exploitation.

D. Aquila Optimization Algorithm (AOA)

The Aquila Optimization Algorithm is a population-based metaheuristic inspired by the hunting strategies of Aquila eagles. It balances global exploration and local exploitation through position updates using a time-varying control factor.

- **Exploration and Exploitation Strategy:** At each iteration, solutions are updated using either a best-guided exploitation move or a random-guided exploration move. The choice is probabilistic, enabling the algorithm to alternate between intensification around high-quality solutions and diversification across the search space.
- **Energy Control Factor:** A decreasing energy term

$$E(t) = 2 \left(1 - \frac{t}{T} \right)$$

controls the transition from global exploration to local refinement, where t is the current iteration number and T is the maximum number of iterations.

- **Local Perturbation:** To further avoid premature convergence, a local neighbor-based waypoint perturbation is applied with a small probability, introducing controlled randomness.
- **Selection Mechanism:** A greedy replacement strategy is used, where newly generated solutions replace current ones only if they achieve a higher fitness value. [12]

E. Greedy Frontier Baseline

To contextualize the metaheuristic results, we employ a deterministic greedy nearest-frontier planner as a baseline. Robots start from distinct boundary positions and iteratively move to the nearest unvisited cell using a Manhattan-distance heuristic until the travel budget D_{max} is exhausted. The method requires no optimization and runs in negligible time, serving as a lower-bound reference for assessing the benefit of metaheuristic approaches.

V. EXPERIMENTAL SETUP

To evaluate the performance and scalability of the proposed algorithms, we developed a simulation environment using Python with NumPy for numerical computations and Matplotlib for trajectory visualization. The experiments were conducted on a standard workstation, measuring solution quality (fitness), computational runtime, and peak memory usage.

A. Test Scenarios

We defined three distinct scenarios of increasing complexity to test the scalability of the algorithms. As the grid size increases, the number of robots and the maximum allowed travel distance (D_{max}) are increased to maintain a balanced workload. The scenarios are detailed in Table I.

TABLE I
EXPERIMENTAL SCENARIOS

Scenario	Grid Size	Robots	Waypoints	D_{max}
A (Small)	10×10	3	8	50.0
B (Medium)	20×20	6	10	100.0
C (Large)	30×30	9	12	150.0

B. Environment Parameters

Across all scenarios, the safety threshold for collision avoidance was set to $d_{safe} = 1.5$ units. The fitness function weights were fixed at $w_1 = 10$ (Coverage), $w_2 = 5$ (Distance), and $w_3 = 2$ (Redundancy) to prioritize area coverage while maintaining reasonable efficiency.

C. Algorithm Configuration

The metaheuristic algorithms were configured with the following parameters based on preliminary tuning:

1) *Simulated Annealing (SA)*: The algorithm employed a geometric cooling schedule with a cooling factor of $\alpha = 0.999$, and ran for 1,000 iterations per trial to allow sufficient exploration of the search space.

2) *Genetic Algorithm (GA)*: We utilized a population size of 50 solutions evolved over 1,000 generations. The mutation operator applied a shift of up to ± 5.0 grid units to randomly selected waypoints to prevent premature convergence.

3) *Particle Swarm Optimization (PSO)*: The swarm consisted of 50 particles evolved for 1,000 iterations. The inertia weight was set to $w = 0.7$, and both the cognitive (c_1) and social (c_2) acceleration coefficients were set to 1.5, balancing the influence of personal bests and global bests.

4) *Aquila Optimization Algorithm (AOA)*: AOA employed a population size of 50 solutions over 1,000 iterations to ensure consistency. A linearly decaying control factor regulated the transition from exploration to exploitation, with a random exploration step factor of $a = 2.0$ and a local perturbation probability of $q = 0.1$.

VI. RESULTS AND DISCUSSION

The four metaheuristic algorithms and the greedy baseline were evaluated across three increasingly scaled scenarios to assess scalability. Due to the stochastic nature of the metaheuristics, results are reported as the mean and standard deviation of the best fitness over 10 independent runs, while the deterministic greedy baseline exhibits zero variance. Table II summarizes the statistical performance of each method.

Overall, AOA achieves the highest mean fitness in the smallest environment, while GA demonstrates the strongest scalability and robustness as problem size increases. All metaheuristic algorithms outperform the greedy baseline.

A. Small Scale Environment (Scenario A)

In Scenario A (10×10 , 3 robots), AOA achieved the highest mean best fitness (Table II), demonstrating the effectiveness of its exploration-exploitation switching in small, constrained environments. GA showed comparable mean performance with

TABLE II
MEAN AND STANDARD DEVIATION OF BEST FITNESS AND RUNTIME OVER 10 INDEPENDENT RUNS

Scenario	Algorithm	Mean Fitness	Std. Dev.	Time (s)
A (10×10)	GA	3.183	0.115	10.9 ± 1.0
	SA	2.342	0.407	2.1 ± 0.2
	PSO	2.810	0.432	13 ± 5
	AOA	3.310	0.354	17 ± 6
	Greedy	-0.637	—	< 0.1
B (20×20)	GA	1.057	0.350	55.4 ± 16
	SA	-0.936	1.547	9.6 ± 2.6
	PSO	-0.589	0.717	62 ± 19
	AOA	-1.988	1.126	67 ± 30
	Greedy	-13.289	—	< 0.1
C (30×30)	GA	-3.318	1.651	189 ± 88
	SA	-6.390	2.648	31 ± 12
	PSO	-7.709	2.301	148 ± 38
	AOA	-11.590	3.468	201 ± 45
	Greedy	-34.425	—	< 0.1

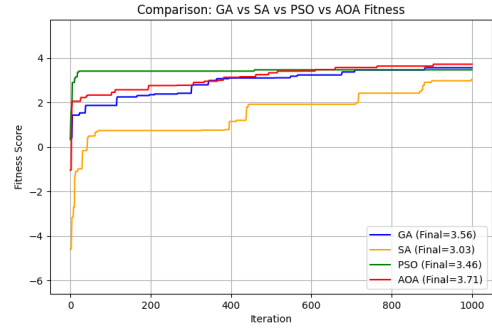


Fig. 1. Convergence of best fitness over iterations for all algorithms in Scenario A (10×10 grid).

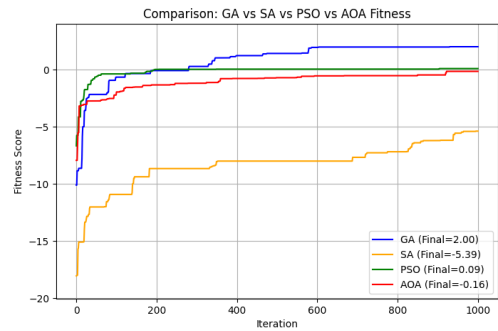


Fig. 2. Convergence of best fitness over iterations for all algorithms in Scenario B (20×20 grid).

lower variance, indicating greater consistency across runs. PSO achieved moderate fitness, while SA recorded the lowest average performance (Table II). As shown in Fig. 1, AOA converges rapidly to high-quality solutions, whereas GA exhibits smoother, more stable convergence. The representative trajectory in Fig. 4 further highlights AOA's efficient exploration and feasible robot coordination.

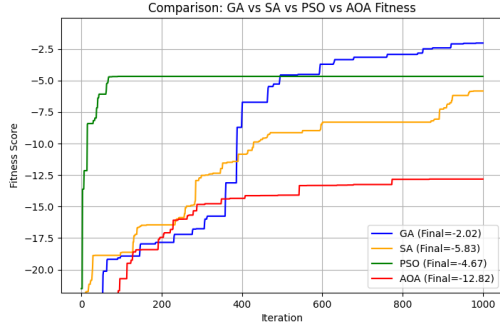


Fig. 3. Convergence of best fitness over iterations for all algorithms in Scenario C (30×30 grid).

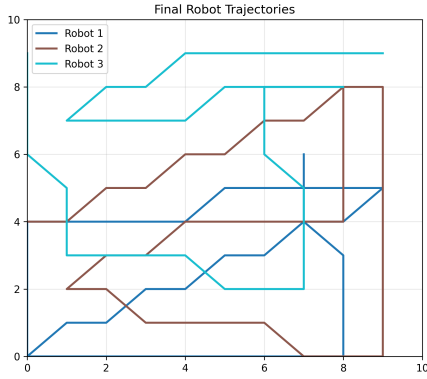


Fig. 4. Representative robot trajectory generated by AOA in Scenario A (10×10 grid), corresponding to the run with the best final fitness among 10 independent executions.

B. Medium Scale Environment (Scenario B)

As complexity increases in Scenario B (20×20 , 6 robots), GA clearly outperforms the other methods, achieving the highest mean fitness (Table II). As shown in Fig. 2, GA exhibits steady convergence, indicating strong robustness under increased coordination and constraint pressure. In contrast, SA, PSO, and AOA yield negative mean fitness values due to frequent penalty dominance, with SA and AOA showing high variance and unstable performance at this scale. The representative GA trajectory in Fig. 5 further confirms coherent path planning in the more complex environment.

C. Large Scale Scalability (Scenario C)

Scenario C (30×30 , 9 robots) highlights significant scalability challenges for all methods. GA achieves the least negative mean fitness (Table II), outperforming the other algorithms and maintaining the lowest variance among large-scale solutions.

SA and PSO suffer substantial degradation, indicating strong penalty dominance. AOA exhibits the weakest scalability with the highest variance. This negative dominance is not due to poor exploration, but rather overly rigid penalty scaling; as grid size and robot count increase, stochastic shifts SA, AOA and continuous velocity updates PSO inherently push waypoints into penalized collision or boundary zones

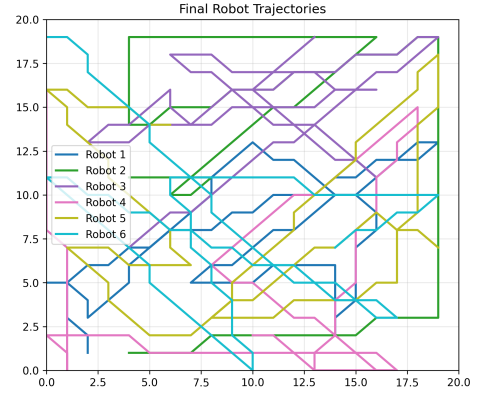


Fig. 5. Representative robot trajectory generated by GA in Scenario B (20×20 grid), selected from the run with the highest final fitness among 10 independent executions.

faster than their coverage fitness can improve. These trends are further reflected in the convergence behavior shown in Fig. 3.

The representative trajectory depicted in Fig. 6 highlights GA's relative robustness at large scale, despite the increased penalty dominance observed across all algorithms.

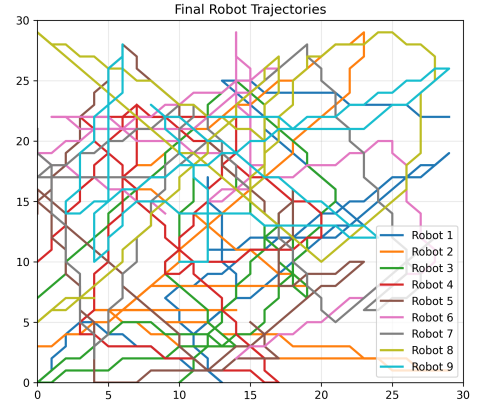


Fig. 6. Representative robot trajectory generated by GA in Scenario C (30×30 grid), illustrating the run with the best final fitness among 10 independent executions.

D. Qualitative and Efficiency Observations

From a computational perspective, Figs. 7 and 8 compare the mean runtime and peak memory usage for the smallest and largest problem scales. In Scenario A, SA exhibits the lowest runtime and memory usage due to its trajectory-based nature, while GA maintains moderate runtime with low and stable memory consumption despite being population-based. PSO and AOA require higher memory and runtime even at small scale.

In Scenario C, all algorithms experience an increase in runtime as a result of the expanded search space and stricter coordination constraints. However, clear differences emerge in scalability: GA maintains comparatively low memory usage while achieving the best solution quality, whereas PSO

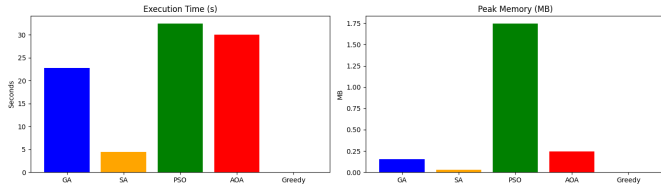


Fig. 7. Runtime and peak memory usage comparison for Scenario A (10×10 grid).

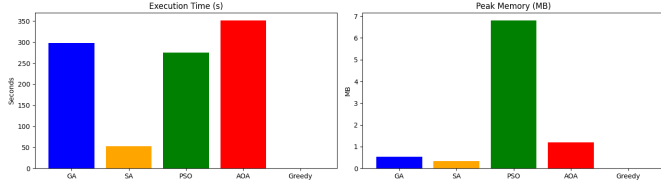


Fig. 8. Runtime and peak memory usage comparison for Scenario C (30×30 grid).

incurs the highest memory consumption and AOA exhibits the longest runtimes. These results indicate that, although SA remains computationally lightweight, its solution quality degrades at large scale, while GA offers a more favorable balance between computational cost and optimization performance for large-scale multi-robot coordination.

E. Limitations and Real-World Considerations

The evaluation assumes perfect localization, constant velocity, static known obstacles, and lossless communication, enabling controlled analysis of algorithmic behavior. In real disaster settings, however, localization errors, dynamic debris, variable speeds, and intermittent communication may affect path feasibility, constraint satisfaction, and overall performance. Thus, while the results offer insights into scalability and coordination, future work should validate these methods through sensor-realistic simulations or hardware-in-the-loop experiments prior to operational deployment.

VII. CONCLUSION AND FUTURE RECOMMENDATIONS

This study compared SA, GA, PSO, and AOA for multi-robot exploration in grid-based disaster environments across three scales (10×10 , 20×20 , 30×30), with performance measured over 10 independent runs.

Beyond algorithm rankings, the results provide transferable insights for practitioners. GA's scalability advantage stems from its population diversity mechanisms—segment-swap crossover and elitist selection prevent premature convergence that causes penalty dominance in PSO and AOA at larger scales. Trajectory-based single-solution methods like SA are structurally limited in large search spaces, as they cannot explore multiple regions simultaneously. Population-based algorithms that alternate between global exploration and local refinement (e.g., AOA or PSO) perform well in small-scale deployments but require adaptive parameter control to remain robust at larger scales. Practitioners can therefore select

strategies based on problem size, coordination complexity, and constraint sensitivity.

The negative mean fitness observed for PSO, SA, and AOA in medium and large scenarios is primarily driven by constraint violations rather than low coverage, suggesting that constraint-repair operators could improve performance more effectively than increasing iterations. For practical deployment, we recommend AOA or PSO for small missions (≤ 100 cells, ≤ 4 robots), GA for medium deployments (100–600 cells, 5–8 robots), and GA with adaptive penalties for large operations (> 600 cells, > 8 robots). Fixed parameter settings were used to ensure fair comparison; adaptive tuning is expected to further enhance scalability. Future work will investigate dynamic parameter adjustment, adaptive penalties, and realistic sensor-noise environments.

REFERENCES

- [1] O. Bushnaq, D. Mishra, E. Natalizio, and I. F. Akyildiz, "Unmanned aerial vehicles (UAVs) for disaster management," in *Nanotechnology-Based Smart Remote Sensing Networks for Disaster Prevention*. Elsevier, 2022, pp. 159–188.
- [2] A. H. Ghoneim, A. M. El Soda, H. A. Abdelmawla, K. Elbadawy, L. Tamer, L. Tarek, J. Magdy, and O. M. Shehata, "Metaheuristic optimization for multi-AGV path planning in search and rescue missions," in *2025 15th Int. Conf. Electr. Eng. (ICEENG)*, 2025, pp. 1–6.
- [3] J. P. Queralta, J. Taipalmaa, B. C. Pullinen, V. K. Sarker, T. N. Gia, H. Tenhunen, M. Gabbouj, and T. Westerlund, "Collaborative multi-robot systems for search and rescue: Coordination and perception," *IEEE Access*, vol. 8, pp. 40990–41017, 2020.
- [4] A. Majeed and S. O. Hwang, "A multi-objective coverage path planning algorithm for UAVs to cover spatially distributed regions in urban environments," *Aerospace*, vol. 8, no. 11, p. 343, 2021.
- [5] A. E. Romeh and S. Mirjalili, "Multi-robot exploration of unknown space using combined meta-heuristic salp swarm algorithm and deterministic coordinated multi-robot exploration," *Sensors*, vol. 23, no. 4, 2023.
- [6] P. Ghassemi and S. Chowdhury, "Multi-robot task allocation in disaster response: Addressing dynamic tasks with deadlines and robots with range and payload constraints," *Robot. Auton. Syst.*, vol. 147, p. 103905, 2022.
- [7] D. Paez, J. P. Romero, B. Noriega, G. A. Cardona, and J. M. Calderon, "Distributed particle swarm optimization for multi-robot system in search and rescue operations," *IFAC-PapersOnLine*, vol. 54, no. 4, pp. 1–6, 2021.
- [8] G. S. Kashyap, D. Mahajan, O. C. Phukan, A. Kumar, A. E. I. Brownlee, and J. Gao, "From simulations to reality: Enhancing multi-robot exploration for urban search and rescue," *arXiv preprint arXiv:2311.16958*, 2023, available at arXiv.
- [9] A. Bendahmane and R. Tlemsani, "Unknown area exploration for robots with energy constraints using a modified butterfly optimization algorithm (xBOA)," *arXiv preprint arXiv:2210.14774*, 2022.
- [10] R. Singh, K. Patel, and D. Gupta, "Energy-aware cooperative exploration for heterogeneous multi-robot systems," *Robot. Auton. Syst.*, 2024.
- [11] F. Ahmed *et al.*, "Route to clean cities: Waste collection optimization with UGVs," in *2025 15th Int. Conf. Electr. Eng. (ICEENG)*, 2025, pp. 1–6.
- [12] F. Hashim and K. Hussain, "Aquila optimizer: Variants and engineering applications," *Eng. Appl. Artif. Intell.*, 2023.
- [13] N. Sharma and R. Singh, "Grid-based path planning using Bresenham algorithm for robotic navigation," *Int. J. Robot. Appl.*, 2020.
- [14] Y. Zhang and H. Liu, "Improved simulated annealing algorithm for multi-objective path planning," *Appl. Soft Comput.*, 2022.
- [15] X. Yang and S. Deb, "Genetic algorithms for robot motion planning: A review," *IEEE Access*, 2020.
- [16] J. Del Ser, E. Osaba, and D. Molina, "Particle swarm optimization: Recent advances and applications," *Swarm Evol. Comput.*, 2021.